

Project: Subway Turnstile Controller

TA: Yongzhi Zhang zhangyzh2022@shanghaitech.edu.cn

Project Overview

In 2025, the Shanghai Metro transported 3.71 billion passengers, ranking first nationwide, with an average daily ridership of 10.15 million. Every turnstile in every station silently completes the cycle of "tap card → verify → allow passage → close." Behind this seemingly simple action lies a precise digital logic control system — one that must respond to sensor signals in real time, determine the direction of travel, handle unauthorized entry, and ensure passenger safety under strict timing constraints.

This project requires you to design a **subway turnstile controller** digital logic circuit. **Starting from a basic two-state turnstile, you will progressively add multi-state access control, direction detection, and more,** ultimately building a complete control system that closely resembles real turnstile behavior. The project is divided into **4 parts**, each with **independent module files and testbenches**, where each subsequent part incrementally adds functionality.

Part 0: Preparation

First and foremost, you need to install iverilog and GTKWave. You will use it to run your project, and generate waveform needed for your project report. Please refer to tutorial for installation guide.

Part 1: Two-State Turnstile (20 pts)

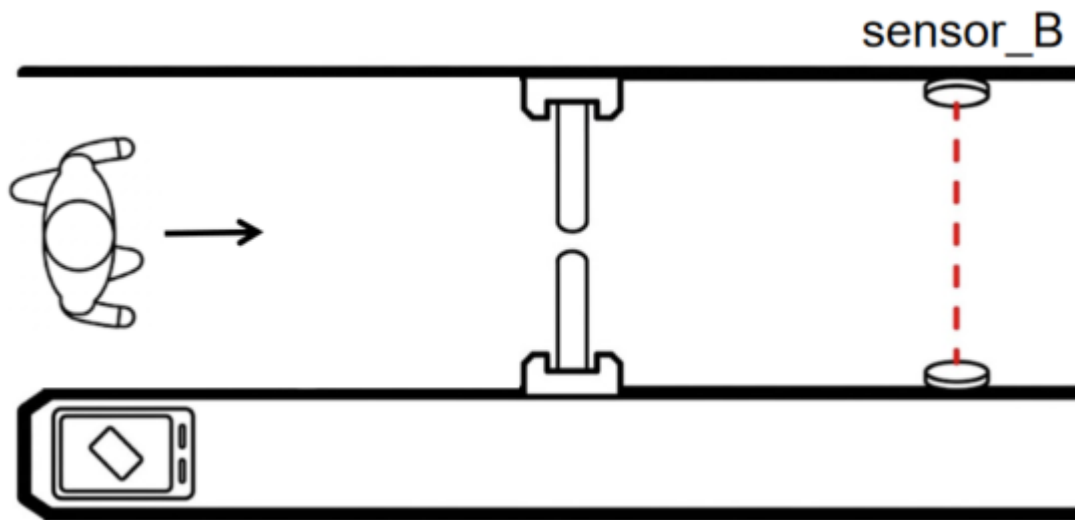
Description

The simplest turnstile works like this: the gate is closed by default; after a passenger taps their card, the gate opens to allow passage; once the passenger walks through, the gate automatically closes and waits for the next person.

If you look closely at this process, the gate is always switching between two states: one where the gate is closed, the red LED is on, and passage is denied; and another where the gate is open, the green LED is on, and passage is allowed. The events that trigger these transitions are also clear: a successful card tap changes the gate from "closed" to "open," and a person passing through changes it from "open" back to "closed." This maps perfectly to a **Moore-type two-state state machine**:

- **LOCKED**: Corresponds to the "gate closed" scenario. Gate closed, red LED on, green LED off, waiting for a card tap. When a valid card signal `card_valid = 1` is received, the system transitions to UNLOCKED.
- **UNLOCKED**: Corresponds to the "gate open" scenario. Gate open, green LED on, red LED off, passage allowed. When the IR sensor detects a person passing through (`sensor_B = 1`), the system transitions back to LOCKED.

Initial state after reset: LOCKED.

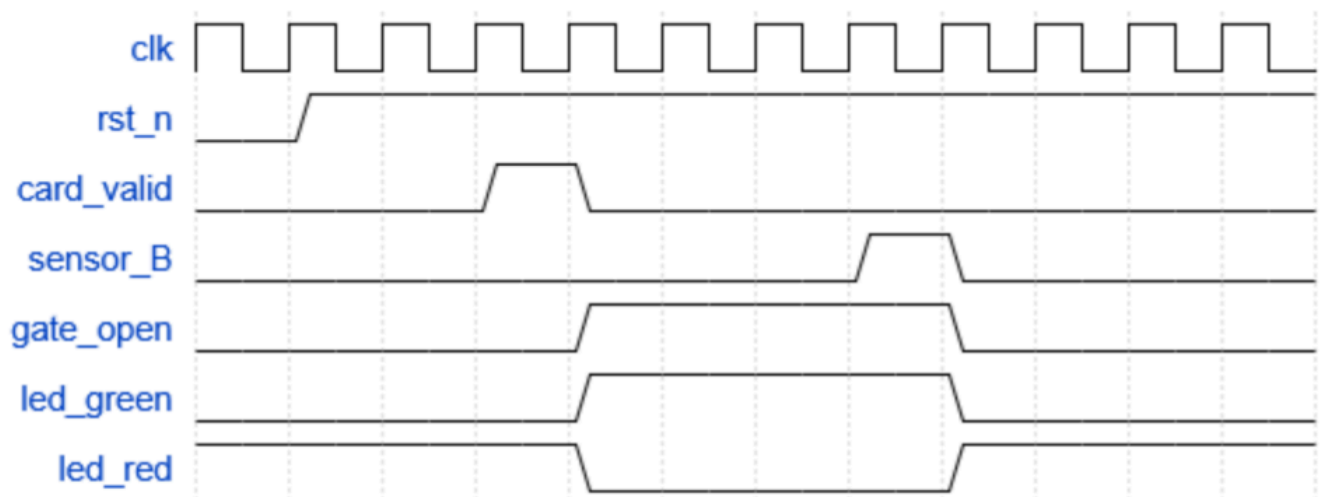


Signal Description

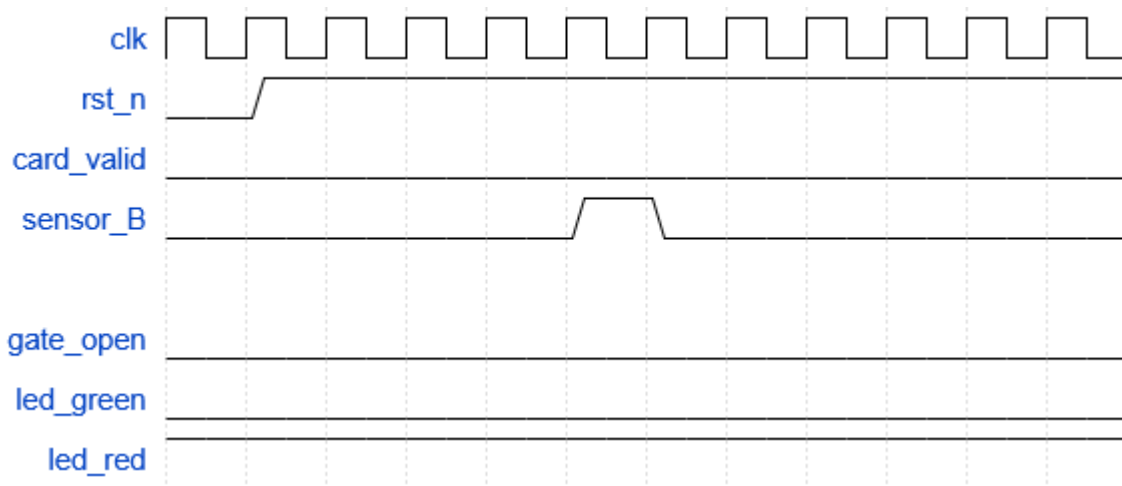
Signal	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Asynchronous active-low reset
card_valid	Input	1	Valid card pulse (1 clock cycle)
sensor_B	Input	1	IR sensor: 1 = person detected
gate_open	Output	1	Gate control: 1 = open
led_green	Output	1	Green indicator (passage allowed)
led_red	Output	1	Red indicator (passage denied)

Reference Waveforms

The following waveform shows the complete sequence: card tap → unlock → person passes → re-lock.



The following waveform shows that when `card_valid` remains 0, the system stays in LOCKED — even if `sensor_B` is triggered, the gate does not open.

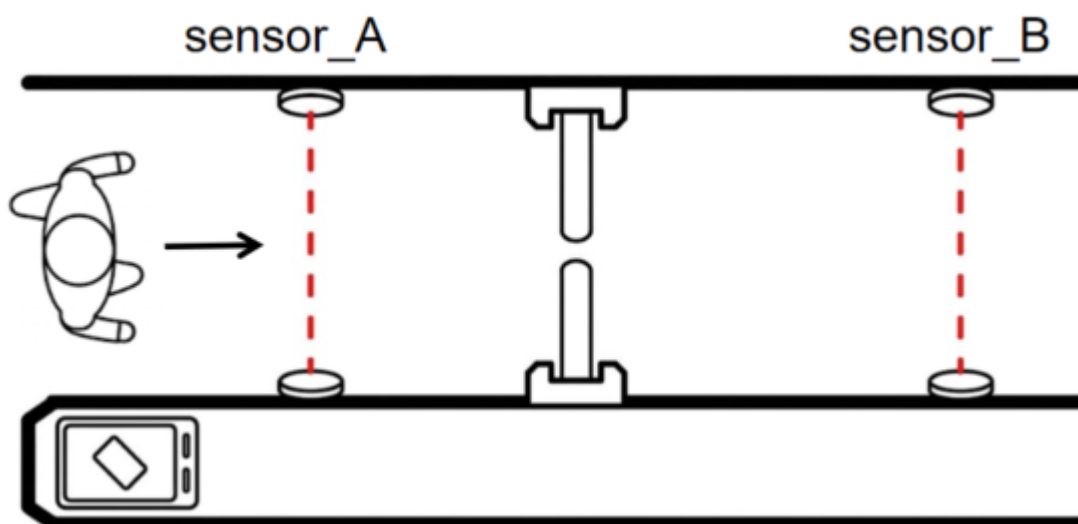


For Part 1, you are only required to implement the code in `part1.v`. The testbench file `part1_tb.v` has been provided for you. Please complete your design accordingly.

Part 2: Basic Access Control (20pts)

Description

The two-state model from Part 1 is clean and simple, but it doesn't quite match how a real turnstile works. In practice, a passenger doesn't just "disappear" after tapping their card — there's a complete process: card tap → gate opens → passenger enters the lane → passenger exits the lane → gate closes. To fully track this process, a single sensor isn't enough — we need an IR sensor at each end of the lane: `sensor_A` (card side, where the passenger enters) and `sensor_B` (platform side, where the passenger exits).



With two sensors, the turnstile's workflow can be broken down into four distinct stages, corresponding to a **four-state Moore-type state machine**:

- **IDLE**: Gate closed, red LED on, system waiting for a card tap. Transitions to VALID upon receiving `card_valid = 1`.
- **VALID**: Card accepted, gate opens, green LED on, waiting for the passenger to enter the lane. Transitions to PASS when `sensor_A = 1` is triggered.

- **PASS:** Passenger has entered the lane, gate remains open, waiting for the passenger to fully pass through. Transitions to CLOSE when `sensor_B = 1` is triggered.
- **CLOSE:** Passenger has passed, gate closes. The controller stays in CLOSE until `sensor_B` returns to 0, then goes back to IDLE on the next clock cycle, ready for the next passenger.

Except for IDLE, `card_valid` is ignored in all other states. Similarly, in the IDLE state, if `sensor_A` is triggered without a prior `card_valid` signal, it should be completely ignored, and the system remains in IDLE.

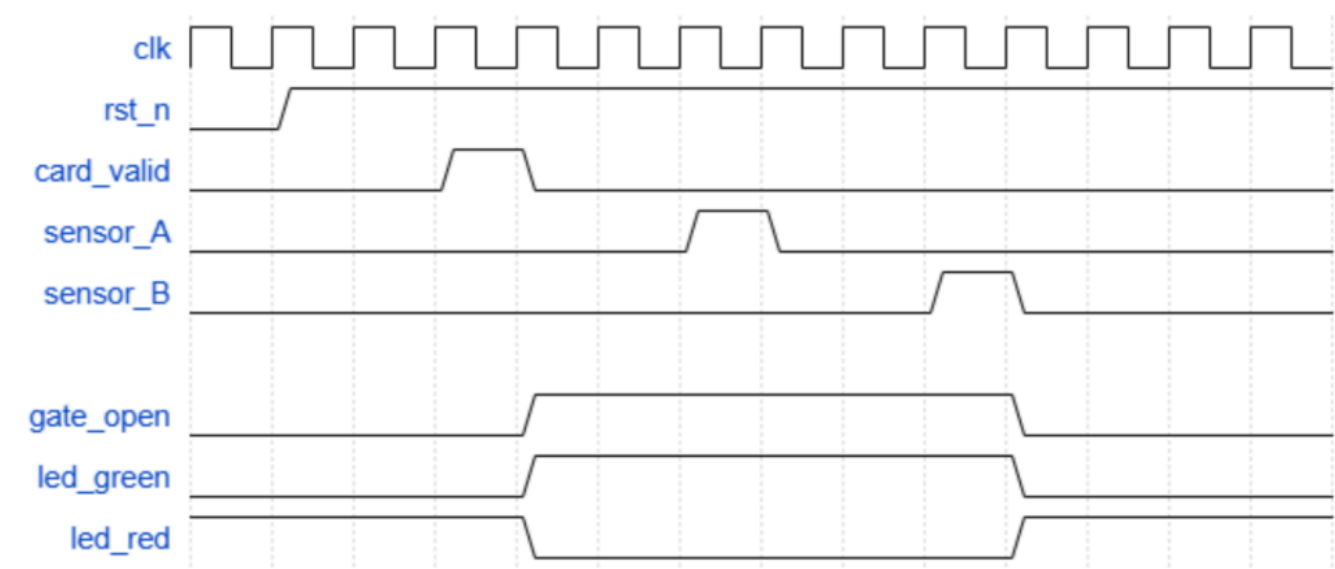
Initial state after reset: IDLE.

Signal Description

Signal	Direction	Width	Description
<code>clk</code>	Input	1	System clock (100 MHz)
<code>rst_n</code>	Input	1	Asynchronous active-low reset
<code>card_valid</code>	Input	1	Valid card pulse (1 clock cycle)
<code>sensor_A</code>	Input	1	Front IR sensor (near card reader): 1 = blocked
<code>sensor_B</code>	Input	1	Rear IR sensor (near platform): 1 = blocked
<code>gate_open</code>	Output	1	Gate control: 1 = open
<code>led_green</code>	Output	1	Green indicator (passage allowed)
<code>led_red</code>	Output	1	Red indicator (passage denied)

Reference Waveforms

The following waveform shows the complete timing of a normal passage:



For Part 2, you are required to complete both `part2.v` and `part2_tb.v`. Your testbench `part2_tb.v` must reproduce the waveforms described above.

Part 3: Reverse Intrusion Detection (20 pts)

Description

Part 2 implemented a complete forward passage workflow, but it hasn't addressed a common real-world problem: what if someone doesn't tap a card and instead forces their way through the turnstile from the platform side?

Recall the positions of the two sensors: `sensor_A` is on the card side, and `sensor_B` is on the platform side. During a normal forward passage, the passenger enters from the card side, so the sensors fire in order: `sensor_A` first, then `sensor_B`. But during a reverse intrusion, someone enters from the platform side — the system is in IDLE (no one tapped a card), yet `sensor_B` suddenly fires. This is clearly an abnormal situation.

We can therefore use a simple rule to detect reverse intrusion: **in the IDLE state, `sensor_B` has higher priority than `card_valid`. If `sensor_B` is triggered, the controller must treat it as a reverse intrusion and enter ALARM on the next clock edge, even if `card_valid` is also high.** When detected, the system should:

- Enter a new **ALARM** state, with the `alarm` output held high for **ALARM_DURATION clock cycles**, then automatically return to IDLE
- During the alarm, the gate remains closed and `led_red` stays on

This means you need to add an ALARM state to the Part 2 state machine and introduce a counter to control the alarm duration.

Signal Description

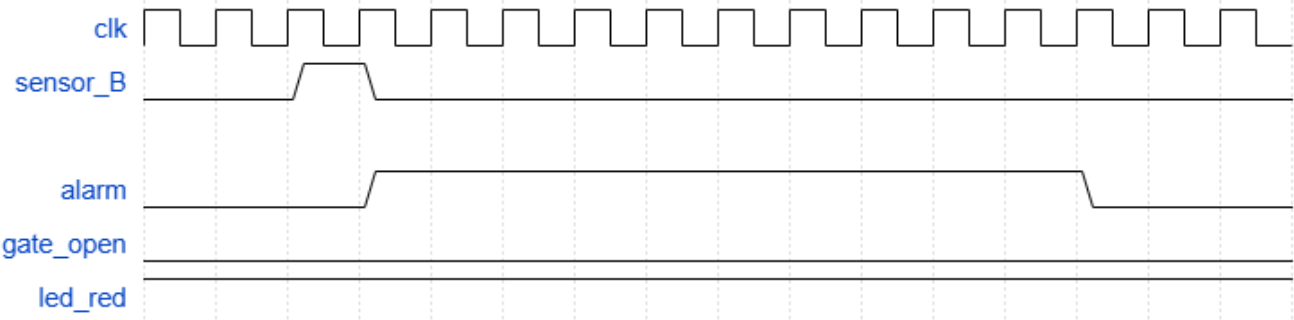
New signal added on top of Part 2:

Signal	Direction	Width	Description
<code>alarm</code>	Output	1	Alarm signal

Reference Waveforms

The following waveform shows the complete timing after `sensor_B` is triggered in IDLE (no `card_valid`), entering the ALARM state.

Note the rising edge and duration of the `alarm` signal (ALARM_DURATION clock cycles). Note that some signals are omitted in the following figure, but you still need to keep them in your code.





For Part 3, you are only required to implement the code in `part3.v`. The testbench file `part3_tb.v` has been provided for you. Please complete your design accordingly.

Here, `ALARM_DURATION` is set to 10 and has already been provided in the code. Please do not modify it, as it will affect your score.

This value is chosen to make the waveform easier to observe in simulation. In real practice, because the clock frequency is very high, the duration should be increased accordingly.

Part 4: Timeout Auto-Close (20pts)

Description

So far, our turnstile keeps the gate open indefinitely after a card tap, waiting for the passenger to enter the lane. But in real life, this often happens: a passenger taps their card and then hesitates, realizes they're at the wrong lane, or turns around to talk to a companion — the gate just stays open, tying up the lane. Real turnstiles solve this simply: set a timeout period, and if no one enters within that time after a card tap, the gate closes automatically.

In terms of state machine design, the focus is on the **VALID** state (card accepted, waiting for the passenger to enter). A timeout counter starts in this state. If the counter reaches `TIMEOUT_DURATION` and `sensor_A` still hasn't been triggered, it means the passenger hasn't entered the lane, so the system should transition to a new **TIMEOUT** state: the gate closes (`gate_open = 0`), the red LED turns on (`led_red = 1`), the green LED turns off (`led_green = 0`), the timeout output goes high (`timeout = 1`), and the system unconditionally returns to IDLE on the next clock cycle.

In the **PASS** state (passenger has entered the lane, waiting for the passenger to exit), another timeout counter starts. If the counter reaches `TIMEOUT_DURATION` and `sensor_B` still has not been triggered, it means the passenger may be stuck in the lane. The system should transition to a new **PASS_ALARM** state: `alarm` remains high, `gate_open` remains 1, `led_green` is 0, `led_red` is 1, and `timeout` is 0. When `sensor_B = 1`, the controller transitions to **CLOSE** first, and then returns to IDLE after `sensor_B` is released (CLOSE to IDLE is the same as previous logic).

Signal Description

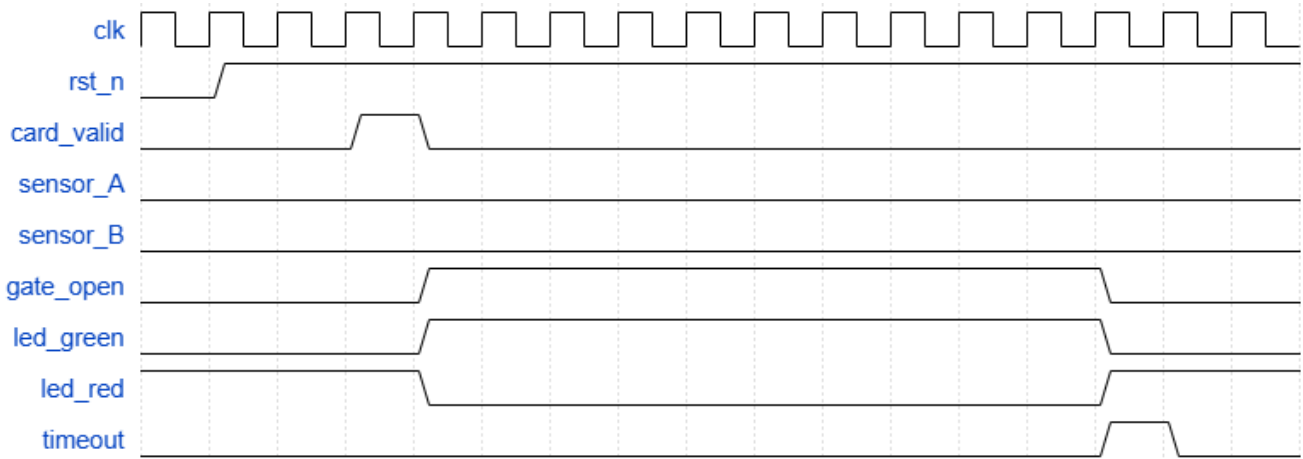
New signal added on top of Part 3:

Signal	Direction	Width	Description
<code>timeout</code>	Output	1	Timeout indicator: 1 = timed out, gate auto-closed

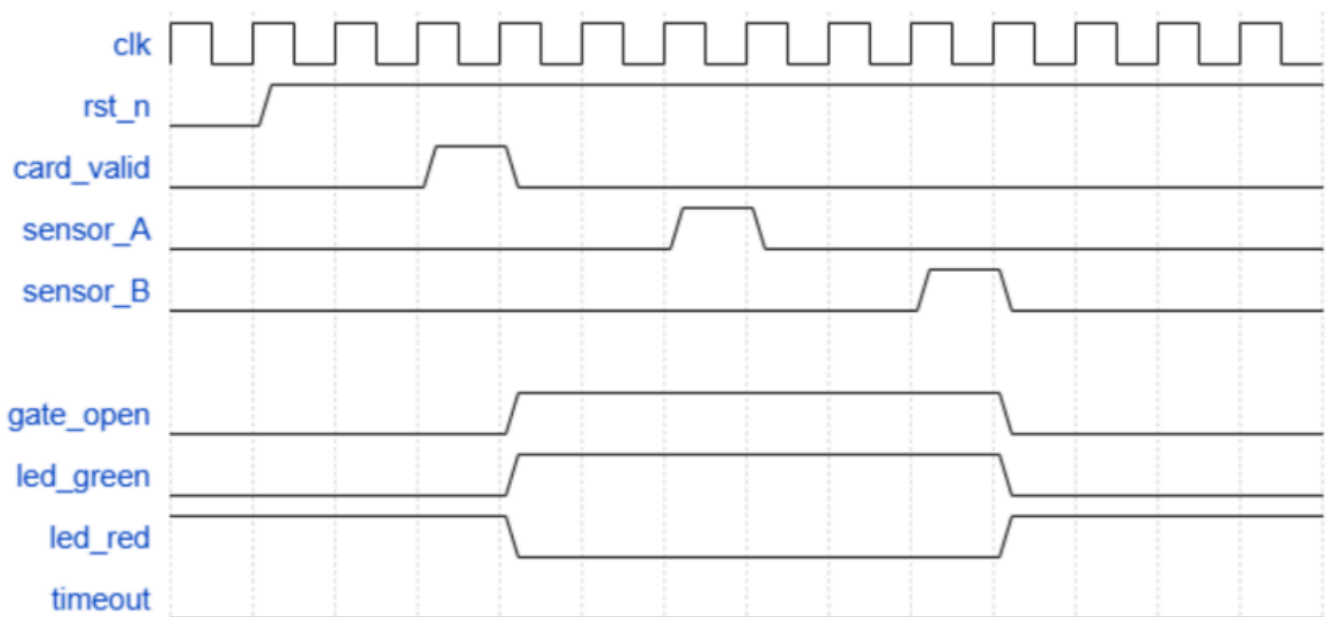
All other signals are the same as Part 3.

Reference Waveforms

The following waveform shows the complete timing when a passenger does not pass after card tap, and the timeout counter reaches the threshold, auto-closing the gate and returning to IDLE.



The following waveform shows a passenger passing through before the timeout — `timeout` stays 0, verifying that normal passage is unaffected by the timeout logic.



For Part 4, you are required to complete `part4.v` and **two** testbench files: `part4a_tb.v` and `part4b_tb.v`.

- `part4a_tb.v` should reproduce the **timeout** waveform (no passage after card tap), i.e., **the first waveform above**.
- `part4b_tb.v` should reproduce the **normal pass** waveform (pass before timeout), i.e., **the second waveform above**.

Like Part 3, `TIMEOUT_DURATION` has been provided in the code. Please do not modify it, as it will affect your score.

For all questions, Do not hardcode numbers for the delay logic. Always use the provided `ALARM_DURATION` and `TIMEOUT_DURATION` parameters in your conditional statements.

Report Requirements

Your `report.pdf` should include the following content:

- The state machine design diagram for each part (Please clearly indicate state names, transition conditions on the arrows, and the specific values of all output signals inside or next to each state node to reflect your Moore FSM design).
- The simulation waveform output for each part (from iverilog and GTKWave).

The report should clearly show how your FSM design matches the required behavior and how the simulation waveforms verify your implementation.

Submission Requirement

Please submit your work as follows:

yourname_yourstudentID.zip (张三_2022123456.zip)

- **part1**
 - part1.v
- **part2**
 - part2.v
 - part2_tb.v
- **part3**
 - part3.v
- **part4**
 - part4.v
 - part4a_tb.v
 - part4b_tb.v
- report.pdf

Example:



Please follow the structure, as it is possible to auto-grade your homework via automated test tools.

Notice

- The provided testbench includes only the most basic test cases. Passing the sample test cases does not guarantee that your design is fully correct. You are therefore expected to devise additional test cases

and write your own testbench to verify the correctness of your code.

- **Plagiarism is strictly prohibited** (this has been emphasized repeatedly in other courses as well).
- This homework is a programming practice for getting you familiar with verilog. Therefore, **Do not use AI** to write code under any circumstances.
- **Point Distribution (100 pts):**

Component	Deliverables	Points
Part 1: Two-State Turnstile	<code>part1.v</code>	20
Part 2: Basic Access Control	<code>part2.v</code> <code>part2_tb.v</code>	20
Part 3: Reverse Intrusion Detection	<code>part3.v</code>	20
Part 4: Timeout Auto-Close	<code>part4.v</code> <code>part4a_tb.v</code> <code>part4b_tb.v</code>	20
Report	<code>report.pdf</code>	20
Total		100